Part A1: Code Exploration

End of Module Assignment – Launch into Computing (January 2026 B)

Running the Original Scripts

Both scripts were executed using the tutor-provided test input: a list of eight strings representing five unique names with inconsistent capitalisation and deliberate duplicates. The Python script was executed using Python 3.12.3; the JavaScript script was executed using Node.js in a terminal environment.

The outputs from both scripts were identical in outcome, though different in presentation. Python returned a flat list on a single line; Node.js formatted the array across multiple lines due to its default console output behaviour for arrays. More significantly, both scripts returned eight entries rather than five. The deduplication had failed silently.

The root cause was consistent across both languages. Neither script normalised input before deduplication. The `set()` function in Python and the `Set()` constructor in JavaScript both operate on exact string matches. As a result, `"Alice"` and `"alice"` were treated as distinct entries, as were `"BOB"` and `"bob"`. No error was raised; the scripts completed and returned an incorrect result without any indication that a problem had occurred. In a production context — a customer database, an HR system, or a healthcare record — this class of silent failure creates duplicate records for the same individual, with downstream consequences for data integrity and compliance.

Modifications Applied

Two modifications were applied to each script. The complete commented code is submitted alongside this report.

The first modification introduced input validation at the function boundary. Both scripts now verify that the input is of the correct type, non-empty, and that every element is a string. Each check raises or throws a typed, descriptive error immediately, preventing malformed data from reaching the core logic. Greenaway (2023) identifies boundary validation as a foundational practice in robust software development, noting that anticipating unexpected inputs is an essential discipline for professional programmers. Handling failure at the entry point is significantly less costly than tracing incorrect output back through a pipeline after the fact (Havill, 2021).

The second modification addressed the deduplication bug. Each name is normalised to lowercase before the set operation, ensuring that case variants of the same name are correctly identified as duplicates. Downey (2024) notes that Python's built-in string methods, such as `lower()` and `title()`, provide concise, readable operations that map closely to natural language processing tasks. Title case is applied to the output after sorting, resulting in a consistently formatted output. The input of eight strings now correctly produces five unique names.

Brief Comparison

Both scripts implement the same logic and produce identical output after modification. The structural differences are instructive.

Python required fewer lines to achieve the same result. Case normalisation, deduplication, sorting, and title case formatting were each expressed in a single line using built-in string methods and list comprehensions. The `str.title()` method is part of Python's standard library, and it requires no additional code. JavaScript has no

built-in equivalent, so a separate helper function of 6 lines was created. This reflects a characteristic noted by Haverbeke (2024): JavaScript's core language is intentionally lean, delegating many utility operations to developers or third-party libraries, where Python's standard library is deliberately broad. In practice, this difference affects development speed and code volume on text-processing tasks.

Ease of understanding was broadly comparable for simple logic but divided on type checking. Python's `isinstance()` and `raise` syntax read naturally and clearly state intent. JavaScript required `Array.isArray()` rather than a simpler type check, because `typeof []` returns `"object"` — a well-documented historical artefact of the language's original design (Haverbeke, 2024). Both scripts benefited significantly from inline comments. Without them, the original one-line deduplication logic offered no indication of its case-sensitivity limitation — an illustration of Greenaway's (2023) observation that readable code is not merely a stylistic preference but a functional requirement for maintainability.

References

Downey, A.B. (2024) *Think Python*. 3rd edn. Sebastopol, CA: O'Reilly Media. Available at: https://learning.oreilly.com/library/view/think-python-3rd/9781098155421/ (Accessed: 3 April 2026).

Greenaway, J. (2023) *Fundamentals for Self-Taught Programmers: Embark on Your Software Engineering Journey without Exhaustive Courses and Bulky Tutorials* Birmingham: Packt Publishing Ltd. Available at:

https://learning.oreilly.com/library/view/fundamentals-for-self-taught/9781801812115/ (Accessed: 3 April 2026).

Haverbeke, M. (2024) *Eloquent JavaScript*. 4th edn. San Francisco: No Starch Press. Available at: https://learning.oreilly.com/library/view/eloquent-javascript-4th/9781098182502/ (Accessed: 3 April 2026).

Havill, J. (2021) *Discovering Computer Science: Interdisciplinary Problems, Principles, and Python Programming*. 2nd edn. Abingdon: CRC Press. Available at: https://doi.org/10.1201/9781003037149